

# AI Health Agent — Build Plan

Oct 3, 2026

Sequenced delivery roadmap

**Confirmed decisions (this plan):** ORM = **SQLAlchemy + Alembic** · detected\_patterns → UserMemory via a **review step first** (auto-merge later once eval trusts it) · golden dataset **harvested from real Phase 1–2 usage** · delivery is **one phase at a time, reviewed before the next begins**.

## What the document commits to

A conversational diet/exercise tracker for Indian eating habits where a **tool-calling Gemini agent** decides intent (save / edit / confirm / ask), backed by **FastAPI + PostgreSQL**, with **two orchestration loops**: a live per-message agent and a **midnight Celery/Redis pipeline** for end-of-day memory + weekly/monthly reports. Reports and memory are built on a layered pattern-detection pipeline (rules → stats → trend diff → LLM discovery → synthesis). Observability/eval/traceability via **Langfuse + Sentry + a golden dataset**.

The key sequencing insight from the doc itself: *"build and validate the agent locally first, on boring/fast infrastructure"* — prove the agent works before investing in infra depth. The plan honors that ordering.

## Guiding principles for sequencing

- **Data model before agent** — every agent action is a DB write; the schema is the contract.
- **One happy path end-to-end before breadth** — a single message → parsed → saved → confirmed, before confidence tiers, memory, reports.
- **Observability wired at the first LLM call**, not bolted on later (Next Step #1 in the doc).
- **The two loops are independent** — the live agent ships and is validated before the midnight pipeline is built.
- **Reports are last** — they depend on accumulated daily summaries and memory, which don't exist until the agent has been running.

## Phase 0 — Foundations & scaffolding

---

*Goal: a running skeleton you can deploy and observe, before any AI logic.*

- **Step 0.1** — Repo scaffold: `backend/` (FastAPI) + `frontend/` (Next.js). Docker + Docker Compose with three services: API, Postgres, Redis. Confirm `docker compose up` gives a healthy stack locally.
- **Step 0.2** — Wire the ORM: **SQLAlchemy** + **Alembic** (confirmed). Migrations working against Postgres.
- **Step 0.3** — Implement the full **data model** as migrations (all 10 tables): Users, FoodLogs, FoodItems, ExerciseLogs, DailySummaries, Sessions, ConversationMessages, UserMemory, Reports, LLMMonitoringLogs. Include `langfuse_trace_id` and `prompt_version` columns on FoodLog/ConversationMessages/Reports from day one.
- **Step 0.4** — Minimal auth + Users CRUD, **including the stored** `timezone` (critical — every cutoff check depends on it, never UTC).
- **Step 0.5** — CI/CD skeleton: GitHub Actions running lint + migrations + a trivial test. Railway project connected (one image, API vs worker differ only by start command).

**Exit criteria:** empty but complete schema deployed, health endpoint live on Railway, CI green.

## Phase 1 — The live agent, happy path only

---

*Goal: "I ate 2 rotis and dal" → parsed → saved → confirmed in reply. Single confidence tier.*

- **Step 1.1** — **Wire Gemini through the Langfuse-wrapped client first** (Next Step #1). The very first LLM call — food parsing — must emit a trace. No un-traced LLM code.
- **Step 1.2** — Define the agent's **tool schema** (function calling): `log_food`, `log_exercise`, and the write tools. All DB writes go through tools.
- **Step 1.3** — Food parsing: message → FoodItems with macros/nutrients. Indian-food aware. Always store the **raw message** on FoodLog (reprocessability).
- **Step 1.4** — Session model: one Session row per calendar day; today's session is the active chat. Record every turn into ConversationMessages (with tool calls + params).
- **Step 1.5** — Agent reply loop: new message → tool call → write → confirm in reply. The single-decision loop from the architecture diagram, minus confidence branching.

**Exit criteria:** a real conversation logs food and exercise correctly, every call traced in Langfuse, raw messages persisted.

## Phase 2 — Confidence tiers, corrections & write-permission rules

---

*Goal: the agent's actual decision-making behavior.*

- **Step 2.1 — Confidence scoring** on each parse → High / Medium / Low, stored on FoodLog. High = save + confirm; Medium = save + flag for correction; Low = ask a clarifying question before saving.
- **Step 2.2 — Conversational corrections** ("actually 3 rotis") and **undo** ("undo that") handled in-session — not separate endpoints.
- **Step 2.3 — Write-permission check:** strict-midnight cutoff (Option A, no grace), evaluated against the user's stored timezone. Today = log/edit/delete/view; yesterday+ = view-only, locked forever. One date comparison.
- **Step 2.4 — Read-only rendering** of past days as summary views (session model Option C).

**Exit criteria:** all three confidence behaviors verifiable; edits blocked after local midnight; undo/corrections work mid-conversation.

## Phase 3 — Evaluation harness (golden dataset)

---

*Goal: stop eyeballing prompt changes. Do this before memory/reports add more LLM surface.*

- **Step 3.1 — Build the golden dataset:** 20–30 real logged messages with known-correct outputs, **harvested from Phase 1–2 usage** (confirmed).
- **Step 3.2 — Scoring functions** per operation: numeric tolerance (food parsing), exact-match (tool selection), LLM-as-judge (report quality — stubbed until reports exist).
- **Step 3.3 — Run as Langfuse experiments** keyed by `prompt_version`, compared across versions. No prompt change is "done" until run against the set.

**Exit criteria:** an eval command produces per-operation scores comparable across prompt versions.

## Phase 4 — Live (in-conversation) memory

---

*Goal: the agent remembers permanent facts stated mid-conversation.*

- **Step 4.1** — UserMemory read **before every reply** (one row per user, living profile — not an event log).
- **Step 4.2** — `update_user_memory` structured tool with a **fixed field enum** ( `food_preferences` , `usual_meals` , ...) so the agent classifies into known buckets. Rule: "Would this still be true next week?" → write immediately (e.g. "I'm vegetarian").
- **Step 4.3** — Feed memory back into the confidence check so the agent asks fewer questions as the profile fills in.

**Exit criteria:** a stated permanent fact persists and measurably reduces later clarifying questions.

## Phase 5 — Midnight pipeline: jobs + end-of-day summarization

---

*Goal: the second orchestration loop, fully independent of the live agent.*

- **Step 5.1** — Celery Beat + Redis queue + Celery Workers. Same Railway image, worker start command. **One job enqueued per active user** (fault isolation).
- **Step 5.2** — Per-job pipeline: build **DailySummary** (aggregated totals) → run the **end-of-day summarizer** (separate, user-invisible LLM call proposing candidate facts/observations).
- **Step 5.3** — **Code-level persistence gate:** a rule check (not the LLM) decides what's permanent/frequent enough to write — e.g. repetition threshold  $\geq 3$  of 7 days before writing an `eating_pattern` .
- **Step 5.4** — **Idempotency** via `Sessions.memory_summarized` flag so retries/re-runs never double-count toward the 7-day threshold.
- **Step 5.5** — Merge accepted observations into UserMemory.

**Exit criteria:** a day locks at midnight (user tz), job runs once, summary + memory updates land, re-running changes nothing.

## Phase 6 — Weekly/monthly reports (the layered pipeline)

---

*Goal: AI-synthesized behavioural reports. Last, because it needs accumulated history.*

- **Step 6.1** — Fetch layer: 7 or 30 DailySummaries + prior Reports + UserMemory.
- **Step 6.2** — **Rule engine**: fixed threshold checks (e.g. breakfast skipped  $\geq 4/7$ ).
- **Step 6.3** — **Statistical engine**: personal z-score baselines, cross-variable correlation, trend/drift regression (pandas/numpy).
- **Step 6.4** — **Trend diff**: this period vs prior Reports rows.
- **Step 6.5** — **LLM discovery pass**: reads raw logged messages for what rules/stats missed; **must cite specific evidence; claims are code-verified before kept**. The LLM never discovers numeric patterns from numbers alone.
- **Step 6.6** — **Synthesis LLM call**: turns all flags + UserMemory into human-readable insight text.
- **Step 6.7** — Store `detected_patterns` (tagged by source) + `ai_insights` + `chart_data` in Reports — **generated once, never regenerated**.
- **Step 6.8** — Merge new patterns into `UserMemory.eating_patterns` **via a review step** (confirmed — auto-merge deferred until eval trusts it).

**Exit criteria:** a weekly report generates end-to-end from the cron, is immutable on re-view, and charts render.

## Phase 7 — Frontend

---

*Goal: user-facing conversational UI + report views. Can begin in parallel once the Phase 1 API stabilizes.*

- **Step 7.1** — Next.js + TypeScript + Tailwind chat interface against the live agent API.
- **Step 7.2** — Past-day read-only summary views; today's active session.
- **Step 7.3** — Weekly/monthly report views with **Recharts** driven by stored `chart_data`.
- **Step 7.4** — Deploy on Vercel.

## Phase 8 — Hardening: observability, traceability, resilience

---

*Goal: close the three pillars (mostly incremental since Langfuse was wired in Phase 1).*

- **Step 8.1** — Full Langfuse nesting: intent classification → model call → tool call under **one trace per user message**. Native Google GenAI wrapper, OTel bridge where needed.
  - **Step 8.2** — **Sentry** for errors across API and workers.
  - **Step 8.3** — LLMMonitoringLogs populated (tokens, cost, latency, confidence, success) per call.
  - **Step 8.4** — Verify **traceability**: every stored output row is one click from its exact input/prompt version/response/confidence via `langfuse_trace_id`.
  - **Step 8.5** — Start with **Langfuse Cloud** (not self-hosted — avoids the ClickHouse/MinIO stack).
- 

## Deferred (explicitly later, per the doc)

---

- Cloud-native rebuild (EventBridge / SQS / Fargate) as a **separate later phase** once the agent works end-to-end.
- Revisiting the Gemini-vs-Claude vendor choice once usage would actually hit Gemini's free-tier limits.

## Critical path

---

The critical path is **Phase 0 → 1 → 2 → 4 → 5 → 6**. Phase 3 (eval) slots in right after the first LLM call and pays for itself across every later phase. Phase 7 (frontend) and Phase 8 (observability) run alongside once the Phase 1 API is stable. Each phase is delivered and reviewed before the next begins.